

WEEK 10:

Started on: Monday, 18 May 2020, 2:20 PM
State: Finished
Completed on: Monday, 18 May 2020, 2:33 PM
Time taken: 12 mins 48 secs
Name: SAKSHI K. C. 2020 BME

Problem Statement:
Write a program to implement **Linear Search** to find a given element in a list of integers.
The program should traverse the array sequentially and determine whether the element is present or not.

Input Format:
1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).
3. An integer **key** to be searched.

Output Format:
• If the element is found, print:
Element found at location : **i**
location starts from 0
• If the element is not found, print:
Element not found.

Sample Input 1:
5
30 40 50 20 10
20
Sample Output 1:
Element found at location : 3

Sample Input 2:
5
30 40 50 20 10
55
Sample Output 2:
Element not found.

For example:

Input	Result
5 30 40 50 20 10 20	Element found at location : 3
5 30 40 50 20 10 55	Element not found.

Answer: (only regime: 0%)

```
1 #include <iostream>
2
3 int LinearSearch(int a[], int n, int key)
4 {
5     int i;
6     for(i=0; i<n; i++)
7     {
8         if(a[i] == key)
9         {
10             return i;
11         }
12     }
13     return -1;
14 }
15
16 int main()
17 {
18     int n, i, key, ans;
19     cout << "n: ";
20     cin >> n;
21     int a[n];
22     for(i=0; i<n; i++)
23     {
24         cin >> a[i];
25     }
26     cout << "key: ";
27     cin >> key;
28     ans = LinearSearch(a, n, key);
29     if(ans != -1)
30     {
31         cout << "Element found at location : " << ans << endl;
32     }
33     else
34     {
35         cout << "Element not found." << endl;
36     }
37     return 0;
38 }
39
40 int LinearSearch(int a[], int n, int key)
41 {
42     int i;
```

Input	Expected	Got	
5 30 40 50 20 10 20	Element found at location : 3	Element found at location : 3	✓
5 30 40 50 20 10 55	Element not found.	Element not found.	✓

Passed all tests: ✓

➔ Thus, program is executed successfully...

Started on: Monday, 18 May 2020, 2:20 PM
State: Finished
Completed on: Monday, 18 May 2020, 2:33 PM
Time taken: 12 mins 48 secs
Name: SAKSHI K. C. 2020 BME

Problem Statement:
Write a program to implement **Binary Search** to find a given element in a sorted array of integers.
The program should traverse the array sequentially and determine whether the element is present or not.

Important Note:
• The array should be sorted in ascending order.
• A sorted array is provided in the sample input.

Input Format:
1. An integer **n** representing the number of elements.
2. **n** space-separated integers (sorted array elements).
3. An integer **key** to be searched.

Output Format:
• If the element is found, print:
Element found at location : **i**
location starts from 0
• If the element is not found, print:
Element not found.

Sample Input 1:
5
10 20 30 40 50
30
Sample Output 1:
Element found at location : 2

Sample Input 2:
5
10 20 30 40 50
55
Sample Output 2:
Element not found.

For example:

Input	Result
5 10 20 30 40 50 30	Element found at location : 2
5 10 20 30 40 50 55	Element not found.

Answer: (only regime: 0%)

```
1 #include <iostream>
2
3 int BinarySearch(int a[], int n, int key)
4 {
5     int i;
6     int mid;
7     int start = 0;
8     int end = n-1;
9     while(start <= end)
10     {
11         mid = (start + end) / 2;
12         if(a[mid] == key)
13         {
14             return mid;
15         }
16         else if(a[mid] < key)
17         {
18             start = mid + 1;
19         }
20         else if(a[mid] > key)
21         {
22             end = mid - 1;
23         }
24     }
25     return -1;
26 }
27
28 int main()
29 {
30     int n, i, key, ans;
31     cout << "n: ";
32     cin >> n;
33     int a[n];
34     for(i=0; i<n; i++)
35     {
36         cin >> a[i];
37     }
38     cout << "key: ";
39     cin >> key;
40     ans = BinarySearch(a, n, key);
41     if(ans != -1)
42     {
43         cout << "Element found at location : " << ans << endl;
44     }
45     else
46     {
47         cout << "Element not found." << endl;
48     }
49     return 0;
50 }
51
52 int BinarySearch(int a[], int n, int key)
53 {
54     int start = 0;
55     int end = n-1;
56     int mid;
```

Input	Expected	Got	
5 10 20 30 40 50 30	Element found at location : 2	Element found at location : 2	✓
5 10 20 30 40 50 55	Element not found.	Element not found.	✓

Passed all tests: ✓

➔ Thus, program is executed successfully...

Question 1

Correct

Report out of 100

Flag question

Problem Statement:

Write a program to sort a list of integers in **ascending order** using the **Insertion Sort** algorithm.

Insertion Sort works by taking one element at a time and inserting it into its correct position in the already sorted part of the array.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

```
5
30 40 50 20 10
```

Sample Output:

```
10 20 30 40 50
```

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```

1 #include <iostream>
2 using namespace std;
3 void InsertionSort(int a[], int n);
4 int main()
5 {
6     int n, i, a[10];
7     scanf("%d", &n);
8     for(i=0; i<n; i++)
9     {
10         scanf("%d", &a[i]);
11     }
12     InsertionSort(a, n);
13     for(i=0; i<n; i++)
14     {
15         printf("%d ", a[i]);
16     }
17     return 0;
18 }
19 void InsertionSort(int a[], int n)
20 {
21     int i, j, temp;
22     for(i=1; i<n; i++)

```

Input	Expected	Got	
5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

➔ Thus, program is executed successfully...

Question 1

Correct

Marked out of 100

Flag question

Problem Statement:

Write a program to sort a list of integers in **ascending order** using the **Bubble Sort** algorithm.

Bubble Sort works by repeatedly comparing adjacent elements and swapping them if they are in the wrong order. This process continues until the array is fully sorted.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

```
5
30 40 50 20 10
```

Sample Output:

```
10 20 30 40 50
```

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```

1 #include <iostream>
2 void BubbleSort(int a[], int n);
3 int main()
4 {
5     int a[10], i, n;
6     scanf("%d", &n);
7     for(i=0; i<n; i++)
8     {
9         scanf("%d", &a[i]);
10     }
11     BubbleSort(a, n);
12     for(i=0; i<n; i++)
13     {
14         printf("%d ", a[i]);
15     }
16     return 0;
17 }
18 void BubbleSort(int a[], int n)
19 {
20     int i, j, t;
21     for(i=0; i<n-1; i++)
22     {
23         for(j=0; j<n-1-i; j++)

```

Input	Expected	Got	
5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50	✓

Passed all tests! ✓

➔ Thus, program is executed successfully...

Question 1
Correct
Marked out of
1.00
Flag question

Write a program to sort a list of integers in **ascending order** using the **Quick Sort** algorithm.
Quick Sort is a **divide and conquer** technique that works by selecting a **pivot element**, partitioning the array into two subarrays, and recursively sorting them.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:
10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 void quicksort(int a[], int left, int right);
4 int main()
5 {
6     int i, n, a[10];
7     scanf("%d", &n);
8     for(i=0; i<n; i++)
9         scanf("%d", &a[i]);
10    quicksort(a, 0, n-1);
11    for(i=0; i<n; i++)
12        printf("%d ", a[i]);
13    return 0;
14 }
15 void quicksort(int a[], int left, int right)
16 {
17     int i, j, temp, pivot;
18     if(left < right)
19     {
20         pivot = left;
21         i = left + 1;
22         j = right;
```

Input	Expected	Got
✓ 5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50 ✓

Passed all tests! ✓

➔ Thus, program is executed successfully...

Started on: Monday, 25 May 2020, 10:11 PM

State: finished

Completed on: Monday, 25 May 2020, 10:13 PM

Time taken: 22 mins, 30 secs

Name: SAURAVEL C S JUDS-BMUE

Question 1

Correct

Marked out of 1.00

Flag question

Write a program to sort a list of integers in **ascending order** using the **Merge Sort** algorithm.
Merge Sort is a **divide and conquer** algorithm that divides the array into smaller subarrays, sorts them recursively, and then merges the sorted subarrays to produce the final sorted array.

Input Format:

1. An integer **n** representing the number of elements.
2. **n** space-separated integers (array elements).

Output Format:

- Print the sorted array in **ascending order**.

Sample Input:

5
30 40 50 20 10

Sample Output:
10 20 30 40 50

For example:

Input	Result
5 30 40 50 20 10	10 20 30 40 50

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 void mergesort(int arr[], int left, int right);
4 void mergesort(int arr[], int left, int center, int right);
5
6 int main()
7 {
8     int i, n, arr[10];
9     scanf("%d", &n);
10    for(i=0; i<n; i++)
11        scanf("%d", &arr[i]);
12    mergesort(arr, 0, n-1);
13    for(i=0; i<n; i++)
14        printf("%d ", arr[i]);
15    return 0;
16 }
17 void mergesort(int arr[], int left, int right)
18 {
19     int center;
20     if(left < right)
21     {
22         center = (left + right) / 2;
```

Input	Expected	Got
✓ 5 30 40 50 20 10	10 20 30 40 50	10 20 30 40 50 ✓

Passed all tests! ✓

➔ Thus, program is executed successfully...